

Architecture & Design Patterns

OVERVIEW

The service follows a layered design — Controller → Service → Repository → Database — with the LLM abstraction sitting alongside the service layer as an independent, swappable dependency. Controllers handle only HTTP concerns; services own business logic and async orchestration; repositories abstract persistence via Spring Data JPA.

DESIGN PATTERNS USED

01 Strategy Pattern

LlmProvider is defined as the strategy interface, with MockLlmProvider and OpenAiLlmProvider as interchangeable implementations. TicketAnalysisService depends only on the interface and has no knowledge of which concrete strategy is active at runtime.

02 Dependency Injection

Every collaborator — TicketRepository, LlmProvider, and others — is injected through the constructor rather than instantiated with new inside business logic. This is Spring's core mechanism, and it's what turns swapping providers into a configuration change instead of a code change.

03 DTO Pattern

CreateTicketRequest, TicketResponse, AnalysisResponse, and ErrorResponse are all kept separate from the JPA entity (Ticket). This lets the API's public shape evolve independently of the underlying database schema.

04 Repository Pattern

TicketRepository abstracts persistence behind JpaRepository, so the service layer never writes SQL or JPQL directly — aside from a single custom filter query used for pagination.

05 Exception Translation / Centralized Exception Handling

A @RestControllerAdvice (GlobalExceptionHandler) converts internal exceptions — TicketNotFoundException, validation errors, type mismatches — into consistent HTTP responses in one place, rather than scattering try/catch blocks across controllers.

06 Fire-and-Forget Async Dispatch

@Async on analyzeAsync() decouples ticket creation from ticket analysis. The caller does not block waiting for the LLM response — the ticket is persisted and returned to the client immediately, while analysis runs on a background thread and the record is updated once it completes.

07 Retry with Exponential Backoff

@Retryable on the LLM call retries transient failures automatically instead of failing the ticket on the first hiccup. It's scoped to LlmTimeoutException specifically, capped at 3

attempts, with the delay doubling between attempts so retries back off rather than hammering the provider:

```
@Retryable(  
    retryFor = LlmTimeoutException.class,  
    maxAttempts = 3,  
    backoff = @Backoff(delay = 500, multiplier = 2)  
)  
public LlmAnalysisResult analyze(LlmAnalysisRequest request) {  
    ...  
}
```

API DOCUMENTATION

Every endpoint is documented and testable through `springdoc-openapi` — the interactive Swagger UI is exposed at `/swagger-ui.html`, backed by an auto-generated OpenAPI schema at `/v3/api-docs`. Request/response DTOs, validation constraints, and status codes are all derived directly from the controller and DTO annotations, so the documentation stays in sync with the code rather than drifting from a hand-written spec.